

LABORATORIO 1

Ejercicio 2: Clientes Desordenados

Código Java y sección del servidor

Tema	Arreglos desordenados de clientes
Operaciones	Alta, baja, modificación, búsqueda y listado
Archivos	N_Cliente.java, Cliente.java y Servidor.java

Enunciado

Una empresa registra para cada cliente su nombre, teléfono, saldo y estado moroso. El programa permite dar de alta, modificar el estado moroso, dar de baja, listar un cliente y listar todos los registros.

Estructura de archivos

- N_Cliente.java: guarda los datos de un cliente.
- Cliente.java: contiene el arreglo desordenado y las operaciones.
- Servidor.java: conecta la página HTML con la lógica Java.

1. Clase N_Alumno.java

```
public class N_Cliente extends Object {  
    private String nombre;  
    private String telefono;  
    private float saldo;  
    private boolean moroso;  
  
    public N_Cliente() {  
    }  
  
    public N_Cliente(String n, String t, float s, boolean m) {  
        nombre = n;  
        telefono = t;  
        saldo = s;  
        moroso = m;  
    }  
}
```

```
public void SetAsignar(String n, String t, float s, boolean m) {  
    nombre = n;  
    telefono = t;  
    saldo = s;  
    moroso = m;  
}  
  
public String GetNombre() { return nombre; }  
public String GetTelefono() { return telefono; }  
public float GetSaldo() { return saldo; }  
public boolean GetMoroso() { return moroso; }  
  
public void SetNombre(String n) { nombre = n; }  
public void SetTelefono(String t) { telefono = t; }  
public void SetSaldo(float s) { saldo = s; }  
public void SetMoroso(boolean m) { moroso = m; }  
  
@Override  
public String toString() {  
    return nombre + "\t" + telefono + "\t" + saldo + "\t"  
        + (moroso ? "Si" : "No") + "\n";  
}  
}
```

2. Clase Alumno.java

Esta es la clase principal. El arreglo se mantiene ordenado alfabéticamente por el nombre cada vez que se registra un empleado.

```
public class Cliente {  
    static int tam = 0;  
    static N_Cliente V[] = null;  
    static int N = -1;  
  
    public static String InicializarWeb(int nuevoTam) {  
        if (nuevoTam <= 0) return "El tamaño del arreglo debe ser mayor que 0";  
        tam = nuevoTam;  
        V = new N_Cliente[tam];  
        N = -1;  
        return "Arreglo creado correctamente con capacidad para " + tam + " clientes";  
    }  
}
```

```
public static int Buscar(String nombre) {
    int i = 0;
    while (i <= N && !nombre.equalsIgnoreCase(V[i].GetNombre())) i++;
    return i > N ? -1 : i;
}

public static String AltaWeb(String nombre, String telefono, float saldo, boolean moroso) {
    if (V == null) return "Primero debe iniciar el arreglo";
    if (N >= tam - 1) return "No hay espacio en el arreglo";
    if (nombre == null || nombre.trim().isEmpty()) return "El nombre no puede estar vacio";
    if (telefono == null || telefono.trim().isEmpty()) return "El telefono no puede estar vacio";
    if (saldo < 0) return "El saldo no puede ser negativo";
    if (Buscar(nombre.trim()) != -1) return "El cliente ya se encuentra registrado";
    N++;
    V[N] = new N_Cliente();
    V[N].SetAsignar(nombre.trim(), telefono.trim(), saldo, moroso);
    return "Cliente registrado correctamente";
}

public static String ModificarWeb(String nombre, boolean moroso) {
    if (V == null) return "Primero debe iniciar el arreglo";
    if (N == -1) return "No hay clientes registrados";
    int pos = Buscar(nombre);
    if (pos == -1) return "El cliente " + nombre + " no fue encontrado";
    V[pos].SetMoroso(moroso);
    return "Estado moroso modificado correctamente";
}

public static String BajaWeb(String nombre) {
    if (V == null) return "Primero debe iniciar el arreglo";
    if (N == -1) return "No hay clientes registrados";
    int pos = Buscar(nombre);
    if (pos == -1) return "El cliente " + nombre + " no fue encontrado";
    for (int i = pos; i < N; i++) V[i] = V[i + 1];
    V[N] = null;
    N--;
    return "Cliente eliminado correctamente";
}
```

```
public static String ListarUnoWeb(String nombre) {
    if (V == null || N == -1) return "No hay clientes registrados";
    int pos = Buscar(nombre);
    if (pos == -1) return "El cliente " + nombre + " no fue encontrado";
    return "Nombre: " + V[pos].GetNombre()
        + "\nTelefono: " + V[pos].GetTelefono()
        + "\nSaldo: " + V[pos].GetSaldo()
        + "\nMoroso: " + (V[pos].GetMoroso() ? "Si" : "No");
}

public static String ListarTodosWeb() {
    if (V == null || N == -1) return "No hay clientes registrados";
    String lista = "";
    for (int i = 0; i <= N; i++) {
        lista += "Nombre: " + V[i].GetNombre()
            + "\nTelefono: " + V[i].GetTelefono()
            + "\nSaldo: " + V[i].GetSaldo()
            + "\nMoroso: " + (V[i].GetMoroso() ? "Si" : "No")
            + "\n-----\n";
    }
    return lista;
}
}
```

3. Servidor.java

Si ya tienes un servidor para los ejercicios desordenados, agrega esta sección al mismo archivo. Si vas a probar solamente este ejercicio, puedes usar el servidor completo siguiente.

```
servidor.createContext("/desordenado2/iniciar", intercambio -> {

    agregarCORS(intercambio);

    if (esOPTIONS(intercambio)) return;

    try {

        Map<String, String> datos = leerFormulario(intercambio);

        enviarRespuesta(intercambio, 200,

            Cliente.InicializarWeb(Integer.parseInt(datos.get("tam"))));

    } catch (Exception e) {

        enviarRespuesta(intercambio, 400, "Error: " + e.getMessage());

    }

});
```

```
servidor.createContext("/desordenado2/alta", intercambio -> {  
    agregarCORS(intercambio);  
  
    if (esOPTIONS(intercambio)) return;  
  
    try {  
        Map<String, String> datos = leerFormulario(intercambio);  
  
        String respuesta = Cliente.AltaWeb(datos.get("nombre"), datos.get("telefono"),  
            Float.parseFloat(datos.get("saldo")),  
            Boolean.parseBoolean(datos.get("moroso")));  
  
        enviarRespuesta(intercambio, 200, respuesta);  
    } catch (Exception e) {  
        enviarRespuesta(intercambio, 400, "Error: " + e.getMessage());  
    }  
});
```

```
servidor.createContext("/desordenado2/modificar", intercambio -> {  
    agregarCORS(intercambio);  
  
    if (esOPTIONS(intercambio)) return;  
  
    try {  
        Map<String, String> datos = leerFormulario(intercambio);  
  
        enviarRespuesta(intercambio, 200, Cliente.ModificarWeb(datos.get("nombre"),  
            Boolean.parseBoolean(datos.get("moroso"))));  
    } catch (Exception e) {  
        enviarRespuesta(intercambio, 400, "Error: " + e.getMessage());  
    }  
});
```

```
servidor.createContext("/desordenado2/baja", intercambio -> {  
    agregarCORS(intercambio);  
  
    if (esOPTIONS(intercambio)) return;  
  
    try {  
        Map<String, String> datos = leerFormulario(intercambio);  
  
        enviarRespuesta(intercambio, 200, Cliente.BajaWeb(datos.get("nombre")));  
    }  
});
```

```
        } catch (Exception e) {

            enviarRespuesta(interambio, 400, "Error: " + e.getMessage());

        }

    });

servidor.createContext("/desordenado2/listaruno", intercambio -> {

    agregarCORS(interambio);

    if (esOPTIONS(interambio)) return;

    try {

        Map<String, String> datos = leerFormulario(interambio);

        enviarRespuesta(interambio, 200, Cliente.ListarUnoWeb(datos.get("nombre")));

    } catch (Exception e) {

        enviarRespuesta(interambio, 400, "Error: " + e.getMessage());

    }

});

servidor.createContext("/desordenado2/listartodos", intercambio -> {

    agregarCORS(interambio);

    if (esOPTIONS(interambio)) return;

    enviarRespuesta(interambio, 200, Cliente.ListarTodosWeb());

});
```

Rutas que utilizará la página HTML

La sección JavaScript del ejercicio debe enviar los datos con método POST a estas rutas:

Operación	Ruta del servidor	Post
Iniciar Arreglo	/desordenado2/Iniciar	Post
Dar de Alta	/desordenado2/Alta	Post
Dar de Baja	/desordenado2/Baja	Post
Modificar Precio	/desordenado2/Modificar	Post
Listar Uno	/desordenado2/ListarUno	Post
Listar Todos	/desordenado2/ListarTodo	Get

Importante para integrarlo

- Los campos HTML son: tam, nombre, telefono, saldo y moroso.
- El campo moroso debe enviarse como true o false.
- No usa base de datos: los datos existen mientras el servidor Java esté ejecutándose.
- La búsqueda se hace de forma secuencial porque el arreglo es desordenado.

Ejercicio 2

Gestión de clientes

**Abrir ejercicio****Documento**

Arreglos Desordenados: Ejercicio 2

Menú de Opciones

Modo Oscuro

Configurar arreglo

Tamaño del arreglo:

Ejemplo: 5

Iniciar ejercicio**Salir**

Arreglos Desordenados: Ejercicio 2

Menú de Opciones

Modo Oscuro

1. Dar de alta**2. Modificar estado moroso****3. Dar de baja****4. Listar un cliente****5. Listar todos****6. Cerrar**



Arreglos Desordenados: Ejercicio 2

Menú de Opciones

 Modo Oscuro

1. Dar de alta

2. Modificar estado moroso

3. Dar de baja

4. Listar un cliente

5. Listar todos

6. Cerrar

Dar de alta

Nombre completo:

Ingrese el nombre completo

Teléfono:

Ejemplo: 88888888

Saldo:

Ejemplo: 1500.50

Moroso:

No

Guardar



Arreglos Desordenados: Ejercicio 2

Menú de Opciones

 Modo Oscuro

1. Dar de alta

2. Modificar estado moroso

3. Dar de baja

4. Listar un cliente

5. Listar todos

6. Cerrar

Modificar estado moroso

Nombre del cliente:

Nuevo estado moroso:

No



Modificar



Arreglos Desordenados: Ejercicio 2

Menú de Opciones

🌙 Modo Oscuro

1. Dar de alta

2. Modificar estado moroso

3. Dar de baja

4. Listar un cliente

5. Listar todos

6. Cerrar

Dar de baja

Nombre del cliente:

Eliminar

Arreglos Desordenados: Ejercicio 2

Menú de Opciones

🌙 Modo Oscuro

1. Dar de alta

2. Modificar estado moroso

3. Dar de baja

4. Listar un cliente

5. Listar todos

6. Cerrar

Listar un cliente

Nombre del cliente:

Buscar



Lista de Clientes Registrados

[← Regresar](#)[🌙 Modo Oscuro](#)

#	Nombre Completo	Teléfono	Saldo	Moroso
---	-----------------	----------	-------	--------

No hay clientes registrados.

Lista de Clientes Registrados

[← Regresar](#)[🌙 Modo Oscuro](#)

#	Nombre Completo	Teléfono	Saldo	Moroso
1	Denis	12345678	1500.54	No